Data-Driven Optimization

선형모형 모델링 및 Or Tools 복습

2주차

선형모형 모델링 및 Or Tools 연습

- 선형 모형 모델링에 익숙하지 않은 학생들은 문제 → 모델링 부분에 대한 학습 필요
- 프로그래밍에 익숙하지 않은 학생들은 기본 프로그램을 잘 숙지하고 모형이 어떻게 최적화 프로그램으로 구현되는지 반복 학습할 것

예제 1 : 주영이의 과일가게

- 과일가게를 운영하는 주영이는 그날 팔고 남은 키위와 딸기로 두 가지 주스를 만들어 판다. ‘새콤 주스’는 키위 4개와 딸기 2개가 필요하고 ‘달콤주스’는 키위 2개와 딸기 6개가 필요하다. 새콤 주스 한 잔을 팔면 500원이 남고 달콤 주스 한 잔을 팔면 400원이 남는다. 어느 날 주영이에게 키위 30개, 딸기 40개가 남았다면 최대 이익을 위해서는 각각 몇 개의 주스를 만들어야 할까?

- 문제의 분석
  - 주영이의 과일가게는 새콤 주스와 달콤 주스를 만들어 팜. → 모든 주스는 팔린다고 가정함.
  - 주스에 따라 다른 재료 구성이 사용되고 이익이 다름.
  - 현재 남아 있는 재료(키위 30개, 딸기 40개)를 가지고 가장 큰 이익을 남기기 위해서는 각 주스를 몇 잔이나 만들어야 하나?

예제 1: 주영이의 과일가게

- 의사결정변수의 파악
  - 이 문제에서 주영이가 결정해야 할 것은?

➔ 최대 이익이 나는 새콤 주스의 개수(x*) 와 달콤 주스의 개수(y*)

- 목적함수의 결정
  - 새콤 주스 한 개당 이익이 500원이고 달콤 주스의 개당 이익은 400원이다. 새콤 주스 수를 x, 달콤 주스 수를 y라 하면 총이익은? ➔ 500x + 400y
- 제약조건의 결정
  - 이익을 최대화 하기 위해서는 가능한 한 많은 주스를 만들면 된다. 그러나 키위 30개와 딸기 40개라는 자원의 한계가 존재한다. 따라서 이 한도 내에서 주스를 만들어야 한다. 이러한 조건은 어떻게 수식으로 나타낼 수 있을까?
  - x개 만큼 새콤 주스를 만들면 새콤 주스는 4개의 키위가 사용되므로 4x의 키위를 새콤 주스 만드는데 사용하게 된다. 마찬가지로 y만큼 달콤주스를 만들면 2y개의 키위가 달콤주스를 만드는데 사용된다. 이렇게 사용된 키위 수는 키위 총량 30개 보다 작아야 하므로 ➔ 4x + 2y <= 30

예제 1: 주영이의 과일가게

- 문장으로 서술된 주영이의 과일가게 문제를 수식으로 정리하면

최대화	$500x + 400y$
제약조건	$4x + 2y \leq 30$
	$2x + 6y \leq 40$
	$x, y \geq 0$

- 해공간
  - 새콤주스 1개, 달콤주스 1개만 만들 수 도 있고 새콤주스 2개, 달콤 주스 2개를 만들 수도 있다. 아니면 새콤주스만 7.5개 또는 달콤주스만 6.67개만 만들 수도 있고 아니면 아예 아무것도 안 만들수도 있다.
  - 이렇게 제약 조건을 만족시키는 (x, y) 즉, 해는 무수히 많이 존재
  - 이렇게 가능한 모든 조합을 일일이 비교해 최적을 조합을 찾아내는 것은 불가능.

선형계획법(Linear Programming)

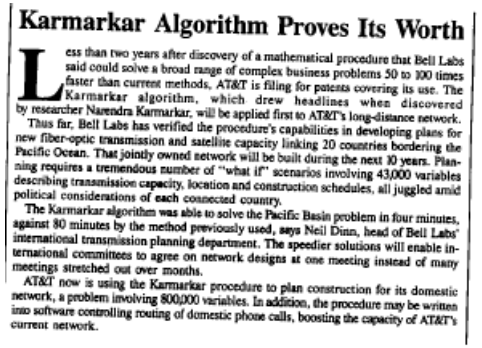
$$\begin{aligned} \max \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \\ & g_i(\mathbf{x}) \le b_i, \qquad x \in \Omega \end{aligned}$$

- 선형계획법
  - x 가 실수, f(x) & g(x) 가 선형(직선 또는 평면)

- Simplex Method
  - George B. Dantzig가 1949년 개발
  - 선형계획법 문제는 해공간(제약조건을 만족하는 해가 존재하는 공간)의 꼭지점에 전역 최적해가 존재함.

    ➔ simplex method는 해공간에서 꼭지점을 효율적으로 이동하며 해를 찾는 방법임

- Interior Point Method
  - Simplex Method와 함께 LP를 푸는 대표적인 방법
  - 최적해를 찾는 시간을 줄임



Karmarkar Algorithm Proves Its Worth

Less than two years after discovery of a mathematical procedure that Bell Labs said could solve a broad range of complex business problems 50 to 100 times faster than current methods, AT&T is filing for patents covering its use. The Karmarkar algorithm, which drew headlines when discovered by researcher Narendra Karmarkar, will be applied first to AT&T's long-distance network.

Thus far, Bell Labs has verified the procedure's capabilities in developing plans for new fiber-optic transmission and satellite capacity linking 20 countries bordering the Pacific Ocean. That jointly owned network will be built during the next 10 years. Planning requires a tremendous number of "what if" scenarios involving 43,000 variables describing transmission capacity, location and construction schedules, all juggled amid political considerations of each connected country.

The Karmarkar algorithm was able to solve the Pacific Basin problem in four minutes, against 80 minutes by the method previously used, says Neil Dinn, head of Bell Labs' international transmission planning department. The speedier solutions will enable international committees to agree on network designs at one meeting instead of many meetings stretched out over months.

AT&T now is using the Karmarkar procedure to plan construction for its domestic network, a problem involving 800,000 variables. In addition, the procedure may be written into software controlling routing of domestic phone calls, boosting the capacity of AT&T's current network.

THE STARTLING DISCOVERY BELL LABS KEPT IN THE SHADOWS

Now its breakthrough mathematical formula could save business millions

It happens all too often in science. An obscure researcher announces a stunning breakthrough and achieves instant fame. But when other scientists try to repeat his results, they fail. Fame quickly turns to notoriety, and eventually the episode is all but forgotten.

That seemed to be the case with Narendra K. Karmarkar, a young scientist at AT&T Bell Laboratories. In late 1984 the 28-year-old researcher astounded not only the scientific community but also the business world. He claimed he had cracked one of the thornest aspects of computer-aided problem-solving. If so, his feat would have meant an instant windfall for many big companies. It could also have pointed to better software for small companies that use computers to help manage their business.

Karmarkar said he had discovered a quick way to solve problems so hideously complicated that they often defy even the most powerful supercomputers. Such problems bedevil a broad range of business activities, from assessing risk factors in stock portfolios to drawing up production schedules in factories. Just about any company that distributes products through more than a handful of warehouses bumps into such problems when calculating the cheapest routes for getting goods to customers. Even when the problems aren't terribly complex, solving them can chew up so much computer time that the answer is useless before it's found.

HEAD START. To most mathematicians, Karmarkar's precocious feat was hard to swallow. Because such questions are so common, a special branch of mathematics called linear programming (LP) has evolved, and most scientists thought that was as far as they could go. Sure enough, when other researchers independently tried to test Karmarkar's process, their results were disappointing. At scientific conferences skeptics attacked the algorithm's validity as well as Karmarkar's veracity.

But this story may end with a different twist. Other scientists weren't able to duplicate Karmarkar's work, it turns out, because his employer wanted it that way. Vital details about how best to translate the Igorithm, whose mathematical notations run on for about 20 printed pages, into digital computer code were withheld to give Bell Labs a head start at developing commercial products. Following the breakup of American Telephone & Telegraph Co. in January, 1984, Bell Labs was no longer prevented from exploiting its research for profit. While the underlying concept could not be patented or copyrighted because it is pure knowledge, any computer programs that AT&T developed to implement the procedure can be protected.

Now, AT&T may soon be selling the first product based on Karmarkar's work—to the U.S. Air Force. It includes a multiprocessor computer from Alliant Computer Systems Corp. and a software version of Karmarkar's algorithm that has been optimized for high-speed parallel processing. The system would be installed at St. Louis' Scott Air Force Base, headquarters of the Military Airlift Command (MAC). Neither party will comment on the deal's cost or where the negotiations stand, but the Air Force's interest is easy to fathom.

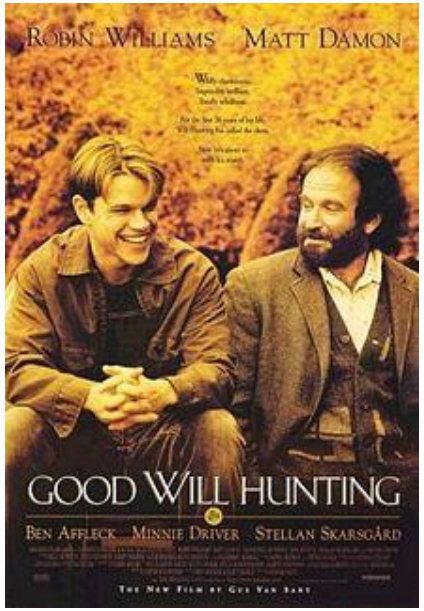
JUGGLING ACT. On a typical day thousands of planes ferry cargo and passengers among air fields scattered around the world. To keep these jets flying, MAC

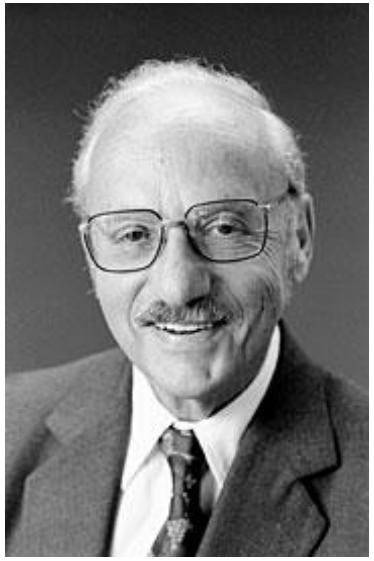
KARMARKAR: SKEPTICS ATTACKED HIS PRECOCIOUS FEAT

BUSINESS WEEK, September 21, 1987

George B. Dantzig

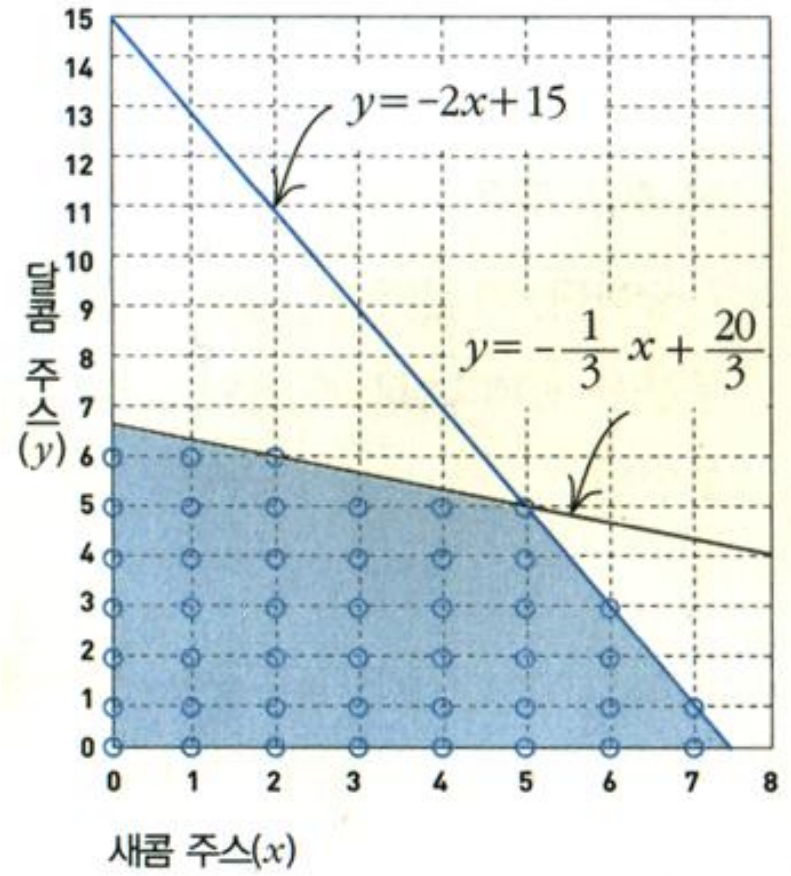
- Good Will Hunting (1998)
  - 어릴 적 상처로 세상과 단절된 상자 안에 자신을 가두려는 천재 윌과 윌의 상처를 치유하여 그 상자에서 끄집어내려는 심리학자 숀 맥과이어의 이야기
  - 맷 데이먼과 로빈 윌리암스
- 통계학의 2대 난제를 푼 천재 대학원생
  - 1939년 UC 버클리, 통계학 수업에 늦은 대학원생. 여느때처럼 칠판 구석에 적혀있는 숙제 두 문제를 일주일간 고민하다 나름 해법을 정리해 숙제를 제출
  - 6주후 일요일 기숙사로 찾아온 교수로 부터 다음과 같은 말을 들음. “자네가 뭘 한 줄 아나! 통계학의 2대 난제를 자네가 푼 거야! 자네가 풀었다고”





예제 1: 주영이의 과일가게

- 제약조건을 그래프로(기하학적으로) 표현하면 꼭지점을 가지는 다차원도형이 됨.



Maximize $500x + 400y$

subject to $4x + 2y \leq 30$

$2x + 6y \leq 40$

$x, y \geq 0$

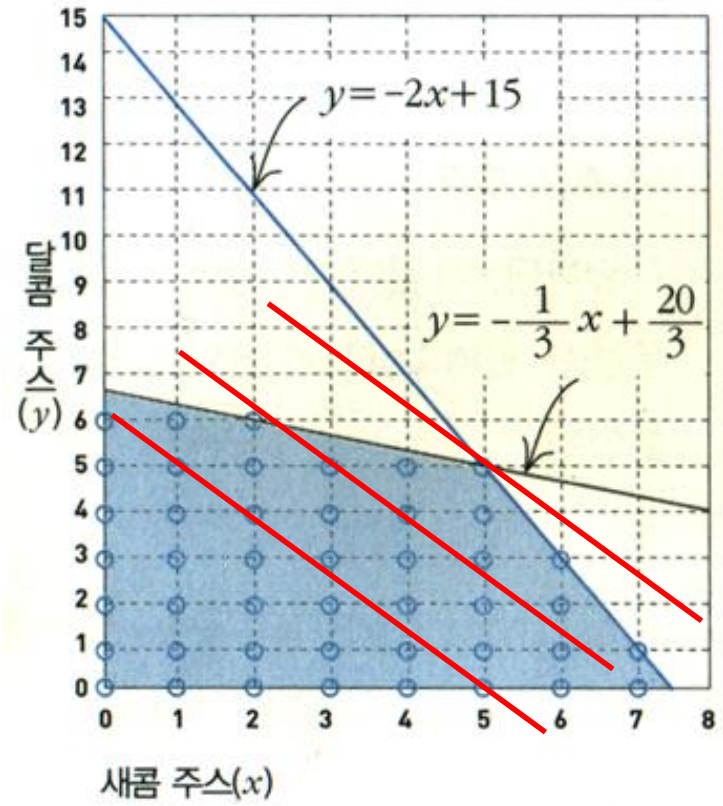
$\Rightarrow$

$y \leq -2x + 15$

$y \leq (-1/3)x + 20/3$

예제 1: 주영이의 과일가게

- 이때 목적함수를 최대화 또는 최소화하는 답은 이들 꼭지점 값 중 하나라는 사실을 발견 → Simplex법의 핵심
- 다시 말해, Dantzig는 선형인 경우 어떤 목적 함수라도 목적 함수의 최적 조건을 만족하는 답은 이 도형의 꼭지점 중에 하나라는 사실을 발견



500x + 400y = 4500

예제 1: 주영이의 과일가게

- Ortools를 이용한 파이썬 코드를 작성하여라.

  (답: 달콤 주스 : 5잔, 새콤주스: 5잔)

- 딸기가 45개 남았을 때 최적해를 구하여라.

  (답: 달콤 주스 : 4.5잔, 새콤주스: 6잔)

프로그램 코드

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver(“SCIP”)

# 의사결정변수



# 제약조건



# 목적함수



status = solver.Solve()
```

```
# 출력
if status == pywraplp.Solver.OPTIMAL:
    print("OPTIMAL")
    print("목적함수값 = %.1f" %(solver.Objective().Value()))
    print('X = %.1f' %(X.solution_value()))
    print('Y = %.1f' %(Y.solution_value()))

else:
    print("The problem does not have an optimal solution.")
```

$$
\begin{aligned}
\text{Maximize} \quad & 500x + 400y \\
\text{subject to} \quad & 4x + 2y \leq 30 \\
& 2x + 6y \leq 40 \\
& x, y \geq 0
\end{aligned}
$$

예제 2

- [예제2.2] 박씨의 식단 문제
  - 비타민 A와 C에 대해서 필요한 1일 최소 요구량 이상을 섭취하여야 함
  - 식품 종류별 비타민 포함량과 가격 (표 2.6)
  - 어떻게 식단을 구성하면 최소비용으로 필요한 비타민을 섭취할 수 있는가?

표 2.6 식품별 비타민 포함량 및 가격

비타민	식품 100 gr에 포함된 비타민 단위 수						1일 필수 요구량
	식품 1	식품 2	식품 3	식품 4	식품 5	식품 6	(단위수)
비타민 A	10	0	20	20	10	20	50(100IU)
비타민 C	0	10	30	10	30	20	60(mg)
식품가격	350	300	500	340	270	400	(원/100gr)

❖ Dantzig 식단문제

예제 2

표 2.6 식품별 비타민 포함량 및 가격

비타민	식품 100 gr에 포함된 비타민 단위 수						1일 필수 요구량
	식품 1	식품 2	식품 3	식품 4	식품 5	식품 6	(단위수)
비타민 A	10	0	20	20	10	20	50(100IU)
비타민 C	0	10	30	10	30	20	60(mg)
식품가격	350	300	500	340	270	400	(원/100gr)

(의사결정변수)

x_i=비타민 식단에 포함되는 식품 i의 포함량 (100g단위) i=1,2,3,4,5,6

(목적함수)

구입 비용 최소화 ⇔ Minimize $350x_1 + 300x_2 + 500x_3 + 340x_4 + 270x_5 + 400x_6 = f(\mathbf{x})$

(제약조건)

(비타민 A 조건) $10x_1 + 20x_3 + 20x_4 + 10x_5 + 20x_6 \geq 50$

(비타민 C 조건) $10x_2 + 30x_3 + 10x_4 + 30x_5 + 20x_6 \geq 60$

(비음 조건) $x_1, x_2, x_3, x_4, x_5, x_6 \geq 0$

$$\begin{aligned} &\max && f(\mathbf{x}) \\ &\text{subject to} \\ &&& g_i(\mathbf{x}) \geq b_i, \quad x \in \Omega \end{aligned}$$

$$g_i(\mathbf{x}) \geq b_i$$

수리모형이 완료된 후

문제점 인식 → 목표 및 제약 확인 → 수리모형 개발 → 해도출 프로그램 개발 → 해 검증

Minimize $350x_1 + 300x_2 + 500x_3 + 340x_4 + 270x_5 + 400x_6$

subject to

$$10x_1 + 20x_3 + 20x_4 + 10x_5 + 20x_6 \geq 50$$

$$10x_2 + 30x_3 + 10x_4 + 30x_5 + 20x_6 \geq 60$$

$$x_1, x_2, x_3, x_4, x_5, x_6 >= 0$$

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver("SCIP")

# 의사결정변수



# 제약조건



(계속)
```

```
(앞에서)
# 목적함수



status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("OPTIMAL")
    print("목적 함수값 = ", solver.Objective().Value())
    print('X1 = %.1f' %(X1.solution_value()))
    print('X2 = %.1f' %(X2.solution_value()))
    print('X3 = %.1f' %(X3.solution_value()))
    print('X4 = %.1f' %(X4.solution_value()))
    print('X5 = %.1f' %(X5.solution_value()))
    print('X6 = %.1f' %(X6.solution_value()))

else:
    print("The problem does not have an optimal solution.")
```

A nicer version

Minimize $350x_1 + 300x_2 + 500x_3 + 340x_4 + 270x_5 + 400x_6$

subject to

$$10x_1 + 20x_3 + 20x_4 + 10x_5 + 20x_6 \geq 50$$
$$10x_2 + 30x_3 + 10x_4 + 30x_5 + 20x_6 \geq 60$$
$$x_1, x_2, x_3, x_4, x_5, x_6 >= 0$$

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver("SCIP")

data = {}
data["constraint_coeffs"] = [
    [10, 0, 20, 20, 10, 20],
    [0, 10, 30, 10, 30, 20]
]
data["supply"] = [50, 60]
data["prices"] = [350, 300, 500, 340, 270, 400]
data["num_vars"] = 6
data["num_constraints"] = 2

infinity = solver.infinity()

#의사결정 변수
x = {}
for j in range(data["num_vars"]):
    x[j] = solver.NumVar(0, infinity, "x[%i]" % j)
```

```
# 제약식
for i in range(data['num_constraints']):
    constraint_expr = [data['constraint_coeffs'][i][j] * x[j] for j in range(data['num_vars'])]
    solver.Add(sum(constraint_expr) >= data['supply'][i])

# 목적함수
objective = solver.Objective()
obj_expr = [data['prices'][j] * x[j] for j in range(data['num_vars'])]
solver.Minimize(solver.Sum(obj_expr))

print(f"Solving with {solver.SolverVersion()}")
status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("Objective value =", solver.Objective().Value())
    for j in range(data["num_vars"]):
        print(x[j].name(), " = ", x[j].solution_value())
else:
    print("The problem does not have an optimal solution.")
```

예제 3: Chess Snackfoods Co.

- Chess Snackfoods Co.는 4가지 상표의 혼합 견과류 제품을 판매하고 있다. 4가지 제품은 Pawn, Knight, Bishop, 그리고 King이다. 각 제품은 특정 비율의 땅콩(peanut)과 캐슈(cashew)를 포함하고 있다. 아래의 표는 각 제품의 1파운드 당 포함되어 있는 두 가지 견과의 양과 각 제품 1 파운드 당 판매가를 보여주고 있다.

	Pawn	Knight	Bishop	King
땅콩(Peanut) (oz.)	15	10	6	2
캐슈(Cashew) (oz.)	1	6	10	14
판매가 ($/lb.)	2	3	4	5

- 이 회사는 매일 750파운드의 땅콩과 250파운드의 캐슈를 공급받기 위해 공급자들과 거래를 하고 있다. 문제는 공급 견과 양을 넘지 않으면서 총 이익을 최대화하기 위해 매일 생산해야 하는 제품의 파운드 양을 결정한다.

예제 3: Chess Snackfoods Co.

(의사결정변수)

(목적함수)

(제약조건)

예제 3: Chess Snackfoods Co. : a nicer version

(모델)

```
from ortools.linear_solver import pywraplp
solver = pywraplp.Solver.CreateSolver("SCIP")
data = {}
data["constraint_coeffs"] = [
    [15, 10, 6, 2],
    [1, 6, 10, 14]
]
data["supply"] = [750, 250]
data["prices"] = [2, 3, 4, 5]
data["num_vars"] = 4
data["num_constraints"] = 2

#의사결정변수
x = {}
for j in range(data["num_vars"]):
    x[j] = solver.NumVar(0, solver.infinity(), "x[%i]" % j)
```

```
#제약 조건
for i in range(data['num_constraints']):
    constraint_expr = [data['constraint_coeffs'][i][j] * x[j]/16 for j in range(data['num_vars'])]
    solver.Add(sum(constraint_expr) <= data['supply'][i])

# 목적함수
objective = solver.Objective()
obj_expr = [data['prices'][j] * x[j] for j in range(data['num_vars'])]
solver.Maximize(solver.Sum(obj_expr))

print(f"Solving with {solver.SolverVersion()}")
status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print("Objective value =", solver.Objective().Value())
    for j in range(data["num_vars"]):
        print(x[j].name(), " = ", x[j].solution_value())
else:
    print("The problem does not have an optimal solution.")
```

1파운드 = 16온즈

실습 1 : 스티글러 식단

- 노벨 경제학상 수상자인 조지 스티글러
- 일련의 음식이 주어졌을 때 기본적인 영양 요구를 충족하는 저렴한 방법을 계산
- **이 예제는 다소 난이도가 높으니 일단 구현해 보자.**

영양소 목록

영양소	일일 권장 섭취량
칼로리	3,000칼로리
단백질	70g
칼슘	0.8g
철분	12밀리그램
비타민 A	5,000 IU
티아민 (비타민 B1)	1.8밀리그램
리보플라빈 (비타민 B2)	2.7밀리그램
니코틴산	18밀리그램
아코르브산 (비타민 C)	75밀리그램

실습 1 : 스티글러 식단

Commodities list

Commodity	Unit	1939 price (cents)	Calories (kcal)	Protein (g)	Calcium (g)	Iron (mg)	Vitamin A (KIU)	Thiamine (mg)	Riboflavin (mg)	Niacin (mg)	Ascorbic Acid (mg)
Wheat Flour (Enriched)	10 lb.	36	44.7	1411	2	365	0	55.4	33.3	441	0
Macaroni	1 lb.	14.1	11.6	418	0.7	54	0	3.2	1.9	68	0
Wheat Cereal (Enriched)	28 oz.	24.2	11.8	377	14.4	175	0	14.4	8.8	114	0
Corn Flakes	8 oz.	7.1	11.4	252	0.1	56	0	13.5	2.3	68	0
Corn Meal	1 lb.	4.6	36.0	897	1.7	99	30.9	17.4	7.9	106	0
Hominy Grits	24 oz.	8.5	28.6	680	0.8	80	0	10.6	1.6	110	0
Rice	1 lb.	7.5	21.2	460	0.6	41	0	2	4.8	60	0
Rolled Oats	1 lb.	7.1	25.3	907	5.1	341	0	37.1	8.9	64	0
White Bread (Enriched)	1 lb.	7.9	15.0	488	2.5	115	0	13.8	8.5	126	0
Whole Wheat Bread	1 lb.	9.1	12.2	484	2.7	125	0	13.9	6.4	160	0
Rye Bread	1 lb.	9.1	12.4	439	1.1	82	0	9.9	3	66	0

실습 1 : 스티글러 식단 - code

```
from ortools.linear_solver import pywraplp
from or2_4_data import *
```

클래스넷에서 or2_4_data.py를 다운로드하여 현재 코딩하는 프로그램 파일과 같은 디렉토리에 저장

```
# 솔버 객체 생성
solver = pywraplp.Solver.CreateSolver( " SCIP " )

# 의사결정변수
foods = [solver.NumVar(0.0, solver.infinity(), item[0]) for item in data]

# 솔버 객체에 저장된 변수 개수 출력
print("Number of variables =", solver.NumVariables())
```

Commodities list

Commodity	Unit	1939 price (cents)	Calories (kcal)	Protein (g)	Calcium (g)	Iron (mg)	Vitamin A (KIU)	Thiamine (mg)	Riboflavin (mg)	Niacin (mg)	Ascorbic Acid (mg)
Wheat Flour (Enriched)	10 lb.	36	44.7	1411	2	365	0	55.4	33.3	441	0
Macaroni	1 lb.	14.1	11.6	418	0.7	54	0	3.2	1.9	68	0
Wheat Cereal (Enriched)	28 oz.	24.2	11.8	377	14.4	175	0	14.4	8.8	114	0
Corn Flakes	8 oz.	7.1	11.4	252	0.1	56	0	13.5	2.3	68	0
Corn Meal	1 lb.	4.6	36.0	897	1.7	99	30.9	17.4	7.9	106	0
Hominy Grits	24 oz.	8.5	28.6	680	0.8	80	0	10.6	1.6	110	0
Rice	1 lb.	7.5	21.2	460	0.6	41	0	2	4.8	60	0
Rolled Oats	1 lb.	7.1	25.3	907	5.1	341	0	37.1	8.9	64	0
White Bread (Enriched)	1 lb.	7.9	15.0	488	2.5	115	0	13.8	8.5	126	0
Whole Wheat Bread	1 lb.	9.1	12.2	484	2.7	125	0	13.9	6.4	160	0
Rye Bread	1 lb.	9.1	12.4	439	1.1	82	0	9.9	3	66	0

실습 1 : 스티글러 식단 - code

```
# 제약조건
constraints = []
for i, nutrient in enumerate(nutrients):
    constraints.append(solver.Constraint(nutrient[1], solver.infinity()))
    for j, item in enumerate(data):
        constraints[i].SetCoefficient(foods[j], item[i + 3])

print("Number of constraints =", solver.NumConstraints())

# 목적함수: 가격으로 정규화된 제품 개수의 합을 최소화
objective = solver.Objective()
for food in foods:
    objective.SetCoefficient(food, 1)
objective.SetMinimization()

status = solver.Solve()
```

9개 영양소

0 3 11

Commodities list

Commodity	Unit	1939 price (cents)	Calories (kcal)	Protein (g)	Calcium (g)	Iron (mg)	Vitamin A (KIU)	Thiamine (mg)	Riboflavin (mg)	Niacin (mg)	Ascorbic Acid (mg)
Wheat Flour (Enriched)	10 lb.	36	44.7	1411	2	365	0	55.4	33.3	441	0
Macaroni	1 lb.	14.1	11.6	418	0.7	54	0	3.2	1.9	68	0
Wheat Cereal (Enriched)	28 oz.	24.2	11.8	377	14.4	175	0	14.4	8.8	114	0
Corn Flakes	8 oz.	7.1	11.4	252	0.1	56	0	13.5	2.3	68	0
Corn Meal	1 lb.	4.6	36.0	897	1.7	99	30.9	17.4	7.9	106	0
Hominy Grits	24 oz.	8.5	28.6	680	0.8	80	0	10.6	1.6	110	0
Rice	1 lb.	7.5	21.2	460	0.6	41	0	2	4.8	60	0
Rolled Oats	1 lb.	7.1	25.3	907	5.1	341	0	37.1	8.9	64	0
White Bread (Enriched)	1 lb.	7.9	15.0	488	2.5	115	0	13.8	8.5	126	0
Whole Wheat Bread	1 lb.	9.1	12.2	484	2.7	125	0	13.9	6.4	160	0

```
"""Entry point of the program."""
# Instantiate the data problem.
# Nutrient minimums.
nutrients = [
    ["Calories (kcal)", 3],
    ["Protein (g)", 70],
    ["Calcium (g)", 0.8],
    ["Iron (mg)", 12],
    ["Vitamin A (KIU)", 5],
    ["Vitamin B1 (mg)", 1.8],
    ["Vitamin B2 (mg)", 2.7],
    ["Niacin (mg)", 18],
    ["Vitamin C (mg)", 75],
]
```

실습 1 : 스티글러 식단 - code

```
constraints.append(solver.Constraint(nutrient[1], solver.infinity()))
```

lower bound, upper bound,

$$44.7\ X1 + 11.6\ X2 + \ldots + 6.4\ X77 \ >= \text{nutrient}[i]$$

```
constraints.append(solver.Constraint(-solver.infinity(), nutrient[1], ))
```

$$44.7\ X1 + 11.6\ X2 + \ldots + 6.4\ X77 \ <= \text{nutrient}[i]$$

실습 1 : 스티글러 식단 - code

```
# 해 출력: 각 음식별 구매량 출력
nutrients_result = [0] * len(nutrients)
print("\nAnnual Foods:")
for i, food in enumerate(foods):
    if food.solution_value() > 0.0:
        print("{}: ${}".format(data[i][0], 365.0 * food.solution_value()))
        for j, _ in enumerate(nutrients):
            nutrients_result[j] += data[i][j + 3] * food.solution_value()
print("\nOptimal annual price: ${:.4f}".format(365.0 * objective.Value()))

print("\nNutrients per day:")
for i, nutrient in enumerate(nutrients):
    print(
        "{}: {:.2f} (min {})".format(nutrient[0], nutrients_result[i], nutrient[1])
    )
```

Commodities list

Commodity	Unit	1939 price (cents)	Calories (kcal)	Protein (g)	Calcium (g)	Iron (mg)	Vitamin A (KIU)	Thiamine (mg)	Riboflavin (mg)	Niacin (mg)	Ascorbic Acid (mg)
Wheat Flour (Enriched)	10 lb.	36	44.7	1411	2	365	0	55.4	33.3	441	0
Macaroni	1 lb.	14.1	11.6	418	0.7	54	0	3.2	1.9	68	0
Wheat Cereal (Enriched)	28 oz.	24.2	11.8	377	14.4	175	0	14.4	8.8	114	0
Corn Flakes	8 oz.	7.1	11.4	252	0.1	56	0	13.5	2.3	68	0
Corn Meal	1 lb.	4.6	36.0	897	1.7	99	30.9	17.4	7.9	106	0
Hominy Grits	24 oz.	8.5	28.6	680	0.8	80	0	10.6	1.6	110	0
Rice	1 lb.	7.5	21.2	460	0.6	41	0	2	4.8	60	0
Rolled Oats	1 lb.	7.1	25.3	907	5.1	341	0	37.1	8.9	64	0
White Bread (Enriched)	1 lb.	7.9	15.0	488	2.5	115	0	13.8	8.5	126	0
Whole Wheat Bread	1 lb.	9.1	12.2	484	2.7	125	0	13.9	6.4	160	0
Rye Bread	1 lb.	9.1	13.4	439	1.1	83	0	9.9	3	66	0

실습 내용

- 예제 1, 2, 3 + 실습 + 과제 1
- 과제 1 은 다음 주 수업 전까지 클래스넷에 업로드

과제 1 (~ 3/19 수업전까지)

다음 3 문제에 대한 최적화 모형을 만들고 이에 대한 해를 찾는 파이썬 프로그램으로 작성하여라.

답안은 프로그램 코드만 제출하되 중간에 '#'을 이용한 코멘트를 작성하여 본인이 잘 이해하고 있음을 보여라.

(문제 1) 경기도 인천 지역의 한 정유소에서는 2종류의 원유를 정제한다. 사우디산 원유와 베네수엘라산 원 유를 증류하여 가솔린, 제트유 및 윤활유를 얻는다. 두 원유는 화학 구성비가 달라 생산되는 제품의 구 성비도 달라진다. 사우디산 원유 1 배럴로부터는 0.3배럴의 가솔린, 0.4 배럴의 제트유, 0.2 배럴의 농을 윤활유가 얻어진다. 한편, 베네수엘라산 원유 1 배럴로부터는 0.4배럴의 가솔린, 0.2 배럴의 제트유 0.3 배럴의 윤활유가 얻어진다.

사우디산 원유는 하루 최대 9000 배럴까지 구입 할 수 있고, 배럴당 가격은 $60 이다. 베네수엘라산 원유는 하루 최대 6000 배럴까지 구입할 수 있고, 배럴당 가격은 $55 이다. 이 정유소에서는 매일 2000 배럴의 가솔린, 1500 배럴의 제트유 및 500 배럴의 윤활유를 공급해야 한다. 이들을 가장 값싸 게 공급할 수 있도록 원유 구입 계획을 수립하라.

과제 1 (~ 3/19 수업전까지)

(문제 2) (주)가홀툴광물은 호주에서 원석을 처리하여 희 귀금속인 가돌리늄(Gd), 홀룸(Ho), 툴륨(Tm)을 채 취하여 공급한다. 원석 1을 1톤을 처리하면 0.2% 가돌리늄, 0.15% 홀룸, 0.2% 툴륨을 함유하고 있고, 원석 2의 1톤에는 0.3% 가돌리늄, 0.25% 홀륨, 0.1% 툴륨을 함유하고 있다. 원석1은 톤당 10만원, 원석 2는 톤당 15만원 비용이 들고, 가돌리늄 8톤, 홀름 6톤, 툴륨 4톤을 추출하여야 한다. 최소비용으 로 원석을 구입하는 방법을 찾아라.

(문제 3) (주)소아 유모차와 보행기 자전거를 생산하여 판 매하면 대당 30만원, 20만원, 16만원의 이익을 볼 수 있다. 유모차는 기계1, 2, 3에서 각각 8시간, 4시 간, 4시간 작업을 하여야 한다. 보행기는 기계1에서 3시간, 기계2에서 4시간 작업하여야 한다. 자전거 한대를 만드는 데에는 기계1을 3시간, 기계3을 2시 간 소요한다. 한편 (주) 소아에서는 기계1과 기계3 의 남는 시간을 인근의 다른 공장에 임대해 줄 것을 요청 받고 있다. 즉, 유모차 보행기 자전거를 생산하 고 남는 기계시간을 빌려주면 기계1은 시간당 3.5 만원, 기계3은 3만원을 받을 수 있다. (주) 소아 입장에서 이익이 최대가 되는 생산계획과 임대 계획을 세워라. 이때 기계 1, 2, 3의 활용가능시간은 240, 200, 100시간이다.